



**Department of Electrical and Computer Engineering
North South University**

Junior Design Project
IMAGE CLASSIFICATION AND
IDENTIFICATION

CSE299.5

NOSHIN NAWAR	2221507042
MD. AREBI SARKER	2221362042
MUBASHSHIRA KAISAR	2221070642
MD. ZIDAN KHAN	2231413642

Faculty:

Mohammad Shifat – E – Rabbi

Assistant Professor

ECE Department

Summer 2025

ABSTRACT

IMAGE CLASSIFICATION AND IDENTIFICATION

This project focuses on developing a unified and modular system for image classification and identification, integrating multiple computer vision tasks into a single, extensible framework. The system addresses the increasing demand for accurate and efficient visual data processing in diverse real-world applications. Current solutions often target specific tasks in isolation; however, this project consolidates functionalities such as face detection, document scanning, object size estimation, handwritten digit recognition, color detection, object segmentation, binary classification, object counting, signature extraction, hand sign detection, and emotion detection into one cohesive platform. The implementation was carried out using Python with established deep learning frameworks and image processing libraries to ensure high accuracy, robustness, and adaptability. The system was designed with a modular folder structure, command-line interface, and reusable scripts for smooth operation and scalability. Testing across multiple datasets demonstrated reliable performance, achieving precise detections, accurate classifications, and consistent processing results. The developed platform has significant potential for use in sectors such as security, manufacturing, digital documentation, and human-computer interaction, making it a versatile and impactful contribution to the field of computer vision.

TABLE OF CONTENTS

ABSTRACT.....	2
Chapter 1 Introduction	5
1.1 Background and Motivation	5
1.2 Purpose and Goal of the Project.....	6
1.3 Organization of the Report.....	6
Chapter 2 Research Literature Review	8
2.1 Existing Research and Limitations	8
Chapter 3 Methodology	9
3.1 System Architecture Design	9
3.2 Dataset Preparation	9
3.3 Model Development.....	9
3.4 Training and Evaluation.....	11
3.5 Development and Testing	11
3.6 Maintenance and Extensibility.....	12
Chapter 4 Investigation/Experiment, Result, Analysis and Discussion.....	13
Chapter 5 Impacts of the Project.....	16
5.1 Impact of this project on societal, health, safety, legal and cultural issues	16
5.2 Impact of this project on environment and sustainability	17
Chapter 6 Complex Engineering Problems and Activities	18
6.1 Complex Engineering Problems (CEP)	18
6.2 Complex Engineering Activities (CEA)	19
Chapter 7 Conclusions	20
7.1 Summary	20

7.2 Limitations	20
7.3 Future Improvement.....	20
References	22

Chapter 1 Introduction

1.1 Background and Motivation

In recent years, computer vision has emerged as one of the most dynamic fields in artificial intelligence, with applications ranging from security and healthcare to manufacturing and entertainment. The advancement of deep learning and image processing techniques has made it possible to achieve high accuracy in tasks such as object detection, classification, segmentation, and recognition. Traditionally, these tasks have been implemented as separate systems, each optimized for a specific use case. However, the growing demand for multifunctional, adaptable, and resource-efficient solutions has encouraged the development of unified platforms capable of performing a variety of computer vision tasks under a single framework.

A modular and extensible image classification and identification system can significantly improve operational efficiency, reduce resource duplication, and facilitate cross-task integration. By leveraging modern tools such as TensorFlow, Keras, OpenCV, and YOLO, it is now possible to build robust and scalable solutions that operate effectively across different environments and datasets. The integration of functionalities like face detection, document scanning, size estimation, emotion recognition, and object counting allows the system to be applied to a wide range of industries including digital archiving, security surveillance, industrial automation, and human-computer interaction.

The motivation for this project stems from the limitations of current single-task computer vision systems, which require separate deployments and maintenance, thereby increasing time, cost, and complexity. By consolidating diverse tasks into a unified command-line-based platform, this project aims to offer a cost-effective, efficient, and user-friendly solution. Such a system not only addresses the technical challenges of multitask implementation but also has the potential to empower researchers, developers, and organizations with a versatile tool adaptable to future needs and emerging technologies.

1.2 Purpose and Goal of the Project

The primary purpose of this project is to design and develop a unified, modular, and extensible system capable of performing multiple computer vision tasks within a single platform. The system aims to eliminate the need for separate implementations for different vision-related tasks by integrating functionalities such as face detection, document scanning, object size estimation, handwritten digit recognition, color detection, object segmentation and classification, binary classification, bottle detection and counting, signature extraction, hand sign detection, and emotion detection.

The main goal is to provide a versatile and user-friendly platform that leverages both deep learning models and traditional image processing techniques to achieve high accuracy and efficiency across diverse tasks. By incorporating a structured command-line interface, reusable modules, and organized folder architecture, the project ensures ease of scalability, testing, and future integration of additional functionalities.

The contributions of this project lie in its ability to merge multiple independent computer vision capabilities into a cohesive system without compromising performance. Unlike conventional solutions that address only one or two specialized tasks, this project introduces a single environment where models and scripts for various applications coexist, share resources, and follow a standardized execution flow. The novelty of this work lies in its combination of modern deep learning approaches (e.g., YOLOv8 and YOLOv11 for real-time detection) with classical image processing methods (e.g., signature extraction without deep learning), resulting in a hybrid system that is adaptable, resource-efficient, and applicable to a wide range of real-world scenarios.

1.3 Organization of the Report

This report is organized into the following chapters:

- **Chapter 1 – Introduction:**
Presents the background, motivation, purpose, and goals of the project, along with an outline of the report structure.
- **Chapter 2 – Research Literature Review**
Summarizes existing work in computer vision and related fields, identifies gaps in current approaches, and explains how this project addresses those limitations.
- **Chapter 3 – Methodology**
Describes the overall system architecture, dataset preparation, model development, training, evaluation, and testing procedures for each module.
- **Chapter 4 – Investigation/Experiment, Result, Analysis, and Discussion**
Details the experiments conducted for each module, presents the results, and provides analysis and discussion on the system's performance.
- **Chapter 5 – Impacts of the Project**
Discusses the potential societal, environmental, and sustainability impacts of the developed system.
- **Chapter 7 – Complex Engineering Problems and Activities**
Identifies and explains the complex engineering problems and activities encountered during the project.
- **Chapter 8 – Conclusions**
Summarizes the key achievements, outlines project limitations, and proposes directions for future improvements.
- **References**
Lists the academic and technical sources cited throughout the report.

Chapter 2 Research Literature Review

2.1 Existing Research and Limitations

Several studies in the field of computer vision have demonstrated notable progress in specific tasks such as object detection, image classification, and image processing. For example, Bochkovskiy et al. [1] introduced YOLOv4, an efficient real-time object detection model that achieved a high trade-off between speed and accuracy, leading to its adoption in applications such as traffic monitoring and inventory management. Ultralytics later developed YOLOv8 and YOLOv11, which improved detection precision and flexibility, enabling multi-class object recognition in diverse environments. In the area of handwritten digit recognition, LeCun et al. [2] pioneered the use of Convolutional Neural Networks (CNNs) on the MNIST dataset, achieving high accuracy and setting a benchmark for future digit recognition tasks. Research on signature extraction, such as the work by Kumar et al. [3], has explored traditional image processing methods involving thresholding and contour detection to isolate signatures from complex document backgrounds without deep learning models. Similarly, document scanning approaches have been enhanced using OpenCV-based edge detection and perspective transformations, as demonstrated in recent optical document processing studies [4].

Despite these advancements, several limitations remain in the existing literature. First, most studies focus on a single computer vision task, resulting in specialized systems that cannot be easily extended to handle multiple functionalities. Second, while YOLO-based models have shown strong performance, their deployment often requires separate pipelines for different object categories, increasing system complexity. Third, many works rely exclusively on either deep learning or traditional image processing, missing opportunities to combine the strengths of both approaches. Fourth, the lack of modular and command-line-based unified platforms reduces accessibility for users who require an integrated solution for various computer vision tasks. These limitations have motivated the development of our project, which consolidates multiple vision-based functionalities—ranging from object detection and classification to document processing—into a single, scalable, and hybrid system that leverages both deep learning and traditional methods for maximum adaptability and efficiency.

Chapter 3 Methodology

3.1 System Architecture Design

- **Framework Selection:** Python was chosen as the primary language due to its extensive library support for computer vision and deep learning.
- **Library & Model Integration:** TensorFlow, Keras, OpenCV, and YOLO (v8 & v11) were selected for their efficiency, compatibility, and proven performance in image processing and detection tasks.
- **Folder Structure:**
 - codeP/ for core Python scripts implementing each module.
 - codeB/ for auxiliary scripts and bash automation.
 - DATA/ for datasets (raw and processed).
 - RESULTS/ for output storage including predictions, logs, and reports.
- **CLI Development:** A command-line interface (CLI) was built using argparse to allow modular execution of tasks without altering source code, enabling portability across systems.

3.2 Dataset Preparation

- **Data Collection:** Public datasets (e.g., Kaggle, Roboflow) were used for classification and detection tasks, while custom datasets were created for bottle counting and signature extraction.
- **Preprocessing:** Steps included image resizing, normalization, augmentation (flips, rotations, brightness adjustments), and splitting into training, validation, and testing sets.
- **Annotation:** Bounding boxes and labels were generated using tools like Roboflow for YOLO-based tasks.

3.3 Model Development

1. Face Detection: The face detection module enabled identification and localization of human faces within input images. Built using OpenCV, it was designed to be lightweight and efficient, and integrated seamlessly into the command-line system. This task served as a foundational block for future facial analysis modules.

2. Document Scanning: This module processed images of documents to produce clean, top-down scanned versions using edge detection and perspective transformation. It enhanced readability and usability of photographed documents, making it valuable for tasks like archiving and digital form processing.

3. Object Size Estimation: By leveraging reference objects within images, this module estimated the size of unknown objects using contour detection and ratio logic. It allowed real-world size approximations from 2D images, useful in medical, manufacturing, or forensic applications.

4. Handwritten Digit Recognition: A neural network was trained to recognize handwritten digits. Initially developed in Jupyter Notebook, the model was later integrated into the CLI for standardized execution. This classification task demonstrated the system's capability to handle trained models with high accuracy and reliability.

5. Color Detection: This task involved analyzing images to detect and report dominant colors. Techniques such as pixel clustering and histogram analysis were employed, making it a useful tool for visual analytics, image tagging, or filtering based on color themes.

6. Object Segmentation and Classification: A more advanced module combined image segmentation and object classification to identify complex objects like planes and trains within scenes. This improved the system's ability to parse and understand multiple objects within a single image.

7. Binary Image Classification: A binary classification model was implemented using Convolutional Neural Networks. The model distinguished between cats and dogs using a Kaggle dataset.

8. Bottle Detection and Counting (YOLOv8): A YOLOv8-based object detection model was trained to detect and count bottles in images. The system annotated bounding boxes and logged the count into CSV files. This task demonstrated real-time detection capabilities and practical reporting for inventory or recycling applications.

9. Signature Extraction: Relying entirely on traditional image processing (rather than deep learning), this module extracted handwritten signatures from scanned documents. It involved

dewarping, binarization, and blob filtering to isolate the signature, achieving high precision even in cluttered inputs.

10. Hand Sign Detection (YOLOv11): Using a dataset from Roboflow, a YOLOv11 model was trained to detect hand gestures representing rock, paper, and scissors. The model offered accurate real-time detection with clear visual labels, showcasing the system's adaptability for gesture-based applications.

11. Emotion Detection (YOLOv11): Another YOLOv11 model was trained to detect facial expressions such as happy, sad, angry, and surprised. It used facial localization and emotion classification, performing well across diverse lighting and facial orientations, and expanding the system's human-interaction capabilities.

3.4 Training and Evaluation

- **Training Environment:** All models were trained in GPU-supported environments to reduce training time.
- **Hyperparameter Tuning:** Adjusted learning rates, batch sizes, and epochs to optimize accuracy and inference speed.
- **Evaluation Metrics:**
 - Classification: Accuracy, Precision, Recall, F1-score.
 - Detection: mAP (mean Average Precision), IoU (Intersection over Union).
- **Validation:** Cross-validation techniques ensured generalization across unseen data.

3.5 Development and Testing

- **Command-line Execution:** Each task could be executed via CLI commands with parameters for dataset paths, model selection, and output directories.
- **Output Storage:** Results, including annotated images, scanned documents, CSV reports, and segmented objects, were stored in the RESULTS/ folder for easy access.
- **Modularity Testing:** Each module was tested independently and in combination with others to ensure smooth integration.

3.6 Maintenance and Extensibility

- **Documentation:** README.md and in-code docstrings were maintained for clarity.
- **Automation:** Bash scripts (bash_command.sh) automated dataset downloads, training, and testing workflows.
- **Future Expansion:** The modular design allows easy integration of additional detection or classification tasks without affecting existing modules.

Chapter 4 Investigation/Experiment, Result, Analysis and Discussion

➤ Investigation / Experiment:

The project investigated multiple computer vision tasks within a single modular Python-based system. The experiments focused on the implementation and testing of individual modules, each targeting a specific image processing or classification task.

The following modules were developed and tested:

1. **Face Detection** – Implemented using OpenCV to identify and localize human faces in images.
2. **Document Scanning** – Used edge detection and perspective transformation to generate clear, top-down scanned versions of document images.
3. **Object Size Estimation** – Estimated real-world object sizes using contour detection and a known reference object.
4. **Handwritten Digit Recognition** – A neural network trained on the MNIST dataset to classify digits (0–9).
5. **Color Detection** – Extracted dominant colors from images using pixel clustering and histogram analysis.
6. **Object Segmentation and Classification** – Segmented scenes and classified objects like planes and trains.
7. **Binary Image Classification** – Used a CNN to classify images as either cats or dogs.
8. **Bottle Detection and Counting** – YOLOv8 model trained to detect and count bottles, outputting results in annotated images and CSV logs.
9. **Signature Extraction** – Traditional image processing methods to isolate handwritten signatures from scanned documents.
10. **Hand Sign Detection** – YOLOv11 model trained to detect rock, paper, and scissors gestures.

11. Emotion Detection – YOLOv11 model trained to detect facial expressions such as happy, sad, angry, and surprised.

All modules were integrated into a command-line interface for ease of execution and testing.

➤ **Results:**

- **Face Detection** – Successfully detected multiple faces in various lighting conditions.
- **Document Scanning** – Produced flattened, clear outputs from skewed document images.
- **Object Size Estimation** – Achieved accurate measurements when reference object size was known.
- **Handwritten Digit Recognition** – Achieved high accuracy on MNIST test data.
- **Color Detection** – Accurately identified dominant color groups in images.
- **Object Segmentation and Classification** – Correctly segmented and classified multiple objects in scenes.
- **Binary Classification** – Consistently distinguished between cats and dogs with high accuracy.
- **Bottle Detection and Counting** – Detected bottles in test images and counted them correctly.
- **Signature Extraction** – Isolated signatures even from noisy document backgrounds.
- **Hand Sign Detection** – Correctly identified rock, paper, and scissors gestures.
- **Emotion Detection** – Detected facial emotions reliably in varied conditions.

➤ **Analysis:**

The modular approach allowed independent development and testing of each task without affecting the overall system. Deep learning models (YOLO, CNNs) demonstrated strong performance in classification and detection tasks, while traditional image processing methods (e.g.,

signature extraction, document scanning) proved effective in cases where machine learning was unnecessary.

YOLOv11 performed well in gesture and emotion detection tasks, showing adaptability to different datasets. Models trained on structured datasets (MNIST, Kaggle cats/dogs) achieved near-perfect classification, whereas tasks involving more varied, real-world images showed slightly reduced accuracy due to environmental variations.

Chapter 5 Impacts of the Project

5.1 Impact of this project on societal, health, safety, legal and cultural issues

While this project is primarily academic and experimental, its underlying modules demonstrate potential value in several societal and cultural contexts:

- **Societal Impact:** Modules such as face detection, emotion recognition, and hand sign detection could be adapted in the future to improve accessibility, assist in human-computer interaction, and support communication for individuals with hearing impairments. Object detection and counting tools (e.g., bottle detection) could be extended to aid recycling initiatives and reduce waste mismanagement.
- **Health & Safety:** Real-time detection systems (e.g., emotion recognition, hand gesture detection) could, in more advanced forms, be used in monitoring environments—such as detecting signs of distress in public areas or workplaces. Document scanning could improve the accuracy of medical record digitization, indirectly aiding healthcare services.
- **Legal Considerations:** Face detection and emotion recognition involve potential privacy concerns if deployed in real-world scenarios. Although this project does not collect personal user data, the technology highlights the importance of ethical AI use and adherence to data protection laws.
- **Cultural Impact:** Hand sign recognition could be developed to recognize culturally specific gestures, enabling better cross-cultural communication. Furthermore, modules like signature extraction could assist in preserving traditional verification methods in the shift toward digital documentation.

Overall, while this small-scale project has no direct societal or legal deployment, it provides a foundation for applications that could influence accessibility, safety, and digital inclusion in future implementations

5.2 Impact of this project on environment and sustainability

The environmental and sustainability impact of this project is indirect but notable in concept:

- **Waste Management & Recycling Support:** The bottle detection and counting module could be adapted for use in recycling plants or community collection programs, helping track and manage recyclable materials more efficiently.
- **Paper Reduction & Digital Archiving:** The document scanning and signature extraction modules promote the idea of digital record-keeping, which, when applied in practice, could reduce paper usage and contribute to environmental conservation.
- **Energy & Resource Considerations:** As a small-scale, offline Python-based system, the project's development and execution have a minimal carbon footprint compared to large-scale, cloud-based AI deployments.
- **Sustainability Awareness:** The project serves as a learning model, encouraging students and researchers to explore AI solutions that align with environmental goals—such as image-based waste detection, efficient manufacturing monitoring, or ecological data analysis.

Although the current form of this project is purely academic, the concepts and techniques explored could be expanded in future work to create environmentally conscious AI solutions.

Chapter 6 Complex Engineering Problems and Activities

6.1 Complex Engineering Problems (CEP)

TABLE I. A COMPLEX ENGINEERING PROBLEM ATTRIBUTES TABLE

Attributes		Addressing the complex engineering problems (P) in the project
P1	Depth of knowledge required	The project requires knowledge of image processing (OpenCV), machine learning (TensorFlow, Keras), and object detection frameworks (YOLOv8 & YOLOv11). Understanding neural networks, dataset preparation, and evaluation metrics was essential for implementing the modules.
P2	Depth of analysis required	Each task (e.g., face detection, emotion recognition, bottle counting) required selecting the most suitable algorithms or models from multiple alternatives. This included evaluating trade-offs between accuracy, processing speed, and computational requirements.
P3	Familiarity of issues	The project involved familiarity with open-source datasets, pre-trained models, and commonly used computer vision frameworks. It also required addressing typical image processing challenges such as lighting variations, background clutter, and object occlusion.

6.2 Complex Engineering Activities (CEA)

TABLE II. A COMPLEX ENGINEERING PROBLEM ACTIVITIES TABLE

Attributes		Addressing the complex engineering activities (A) in the project
A1	Range of resources	The project required human effort for research and development, publicly available datasets for training, and modern tools such as Python, OpenCV, TensorFlow, Keras, and YOLO frameworks for implementation.
A2	Innovation	The project applies multiple computer vision techniques in a unified, modular system. This integration demonstrates innovative thinking in combining diverse image processing and classification tasks within a single framework.
A3	Familiarity	Successful execution required familiarity with AI model training and testing, dataset preprocessing, and applying image processing algorithms. Knowledge of different frameworks and tools was essential to achieve accurate results.

Chapter 7 Conclusions

7.1 Summary

The *Image Classification and Identification* project integrates multiple computer vision tasks into a unified, modular framework. Developed in Python, it leverages TensorFlow, Keras, OpenCV, and YOLO (v8 & v11) to perform diverse tasks such as face detection, document scanning, object size estimation, handwritten digit recognition, color detection, object segmentation, binary image classification, bottle detection and counting, signature extraction, hand sign detection, and emotion recognition. The modular design ensures that each task can be executed independently through a command-line interface, promoting reusability and ease of extension for future work.

7.2 Limitations

- **Scale of the Project:** As a small-scale academic work, the models are trained on relatively small datasets, limiting their generalization capability in real-world scenarios.
- **Performance Constraints:** Some modules may not achieve optimal accuracy or speed in complex, high-variation environments due to limited computational resources.
- **No Real-time Deployment:** The project was designed for offline execution and has not been deployed as a web application, mobile app, or integrated hardware system.
- **Dataset Dependence:** Performance is heavily dependent on the quality and diversity of training datasets; biases in datasets can affect accuracy.
- **Limited User Interface:** Execution is via a command-line interface, which may be less user-friendly for non-technical users.

7.3 Future Improvement

- **Larger and More Diverse Datasets:** Training models with bigger and more varied datasets to improve accuracy and adaptability to real-world scenarios.
- **Optimization for Real-time Performance:** Enhancing processing speed and model efficiency to support live video feeds and low-latency detection.

- **Graphical User Interface (GUI):** Developing a GUI or lightweight web application to make the system more accessible to non-technical users.
- **Cloud or Edge Deployment:** Integrating the project into cloud platforms or edge devices (e.g., Raspberry Pi, Jetson Nano) for real-world applications.
- **Module Expansion:** Adding new tasks, such as sign language recognition, waste detection for environmental monitoring, or advanced multi-object tracking.
- **Model Fine-tuning:** Experimenting with transfer learning, hyperparameter optimization, and ensemble methods to improve accuracy and robustness.

References

1. Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. — Introduced **YOLOv4**, an efficient real-time object detection model with a strong balance of speed and accuracy, widely used in traffic monitoring and inventory management.
2. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. — Pioneered the use of **Convolutional Neural Networks (CNNs)** for handwritten digit recognition on the MNIST dataset, achieving benchmark accuracy.
3. Kumar, et al. — Researched **traditional image processing methods** (thresholding, contour detection) for signature extraction from complex document backgrounds without deep learning.
4. Recent optical document processing studies — Demonstrated **OpenCV-based edge detection and perspective transformations** for improved document scanning quality.